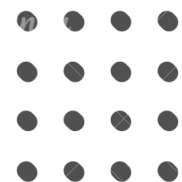




VMES Grinds





Leetcode: 290. Word Pattern
Difficulty – Easy.





290. Word Pattern

Solved

Easy

Topics

Companies

Given a `pattern` and a string `s`, find if `s` follows the same pattern.

Here **follow** means a full match, such that there is a bijection between a letter in `pattern` and a **non-empty** word in `s`. Specifically:

- Each letter in `pattern` maps to **exactly** one unique word in `s`.
- Each unique word in `s` maps to **exactly** one letter in `pattern`.
- No two letters map to the same word, and no two words map to the same letter.

Example 1:



Example 1:

Input: pattern = "abba", s = "dog cat cat dog"

Output: true

Explanation:

The bijection can be established as:

- 'a' maps to "dog".
- 'b' maps to "cat".

Example 2:

Input: pattern = "abba", s = "dog cat cat fish"

Output: false

Example 3:

Input: pattern = "aaaa", s = "dog cat cat dog"

Output: false



Constraints:

- `1 <= pattern.length <= 300`
- `pattern` contains only lower-case English letters.
- `1 <= s.length <= 3000`
- `s` contains only lowercase English letters and spaces `' '`.
- `s` **does not contain** any leading or trailing spaces.
- All the words in `s` are separated by a **single space**.



Brain Storm 3 mins





Intuition





Word Pattern

1. How can I convert this into a list?
2. What can I do next? Starts with small
3. Mapping
 - a. A maps to dog `key['a'] == 'dog'`
 - b. B maps to cat
 - c. What if A maps to fish again `????` -> false
4. Position checking
 - a. How can I get index, value at the same time from a list.
 - b. check with `key(value) → list(value)`



Explanation





Word Pattern



a : dog
b: dog

abba ['dog', 'cat', 'cat', 'fish']

0 -> hashMap['a'] = 'dog'

is it the same as list[0]??? -> True

move on to the next



Code





```
class Solution:
    def wordPattern(self, pattern: str, s: str) -> bool:
        hashMap = {}
        s = s.split()
        if len(pattern) != len(s):
            return False

        for i in range(len(pattern)):
            if pattern[i] not in hashMap:
                if s[i] in hashMap.values():
                    return False
                hashMap[pattern[i]] = s[i]

        for index, value in enumerate(pattern):
            if hashMap[value] != s[index]:
                return False
        return True
```



Time and Space Complexity Analysis





```
class Solution:
    def wordPattern(self, pattern: str, s: str) -> bool:
        hashMap = {}
        s = s.split()  $O(L)$ 
        if len(pattern) != len(s):  $O(n), O(m)$ 
            return False

        for i in range(len(pattern)):  $O(n)$ 
            if pattern[i] not in hashMap:  $O(1)$ 
                if s[i] in hashMap.values():  $O(k)$  where k is the size of the dict
                    return False
                hashMap[pattern[i]] = s[i]  $O(1)$ 

        for index, value in enumerate(pattern):  $O(n)$ 
            if hashMap[value] != s[index]:
                return False
        return True
```

Time Complexity - $O(n^2)$

Space - $O(n)$



Key Takeaway



Takeaway 1: To track key and value pairs -
Hash Map is the choice